

International University - VNU HCMC
FINAL GAME PROJECT REPORT

Project name: “Plant vs Zombie” Game Development

Group: D

Student name: Trần Minh Khoa

Student ID: ITCSIU25025

Student name: Nguyễn Văn Đức

Student ID: ITCSIU25012

Student name: Trần Văn Thiện

Student ID: ITCSIU25049

Student name: Lý Gia Khang

Student ID: ITITI25011

Student name: Trần Thụy Hoàng Trúc

Student ID: MAMAIU25037

TABLE OF CONTENT

1. Overview

- **1.1. General introduction**
- **1.2. Core goal**

2. Gameplay Mechanics

- **2.1. Game resource systems**
- **2.2. Combat systems**
- **2.3. The enemy's attack mechanism**

3. UI/UX Design

- **3.1. UI Design**
- **3.2. UX Design**

4. Core Algorithm And Data Structure Analysis

- **4.1. The lawn**
- **4.2. Lists/Arrays**
- **4.3. Collision detection**
- **4.4. Spawner algorithm**

5. Conclusion

- **5.1. Achieved**
 - **5.2. Challenges and solutions**
-

1. OVERVIEW

1.1. General introduction

“Plants vs Zombies” is a tower defense game combined with resource management. This is a genre that challenges players' calculation and thinking skills to optimize available resources in order to defend their stronghold. Thanks to its captivating gameplay, where players can utilize their strategic skills to achieve victory, this tower defense genre has become incredibly popular in the gaming community.

1.2 Core goal

The objective of the game is for the player to use resources (Sun) to build and arrange defensive units (Plants) to prevent waves of Zombies from attacking the house. Players win by successfully defending against waves of zombies without letting a single zombie get into the house.

2. GAMEPLAY MECHANICS

2.1. Game resource systems

The Sun is the most important resource in the game. Suns are randomly generated by falling from the sky or by sunflowers when players plant them on the battlefield. Each plant requires a certain amount of Sun before it can be planted on the battlefield.

2.2. Combat systems

When plants are planted on the battlefield, each species will have a different combat function (for example, Peashooter will fire bullets horizontally at zombies advancing in the same line as the plant; the bullets will damage zombies, and each burst will require a cooldown). Other species, Walnuts can tank zombie attacks (eating).

Zombies will take damage from attacks by attacking plants and will be slowly killed. Conversely, zombies will stop moving and begin consuming the plants upon contact.

2.3. The enemy's attack mechanism

Zombies will appear in waves and randomly in different rows. They move from right to left of the screen, from outside the arena towards the base, and only plants in the same row as the zombies can attack them or be attacked by them.

3. UI/UX DESIGN

3.1. UI Design

The interface design is optimized so that the speed of left-clicking matches the speed of the player's thinking. The Seed Bank in the upper corner and the Sun counter assist in timing and arranging the battlefield for the player. The Seed Bank and the Sun counter are placed side-by-side for easy comparison.

3.2. UX Design

In the game, players are placed on a 9x5 (9 columns and 5 rows) arena. The selection status of plant cards is indicated by visual feedback; if a card is selected, it is highlighted with a dark shadow to let the player know which plant is currently in hand. The zombie's health bar is hidden and is calculated by decreasing the number of bullets they take at the start of the wave, for example, a normal bullet reduces 20 HP until the zombie dies.

4. CORE ALGORITHM AND DATA STRUCTURE ANALYSIS

4.1. The lawn

Instead of a traditional 2D matrix, the map is represented using Object-Oriented Programming principles with an array of 5 independent "Lane" objects. Each lane manages its own grid intersections to form planting spaces. Each empty space can only contain one type of plant; if a player wants to plant a different type, they must use the Shovel item to remove the old plant.

4.2. Lists/Arrays

Use a list (or dynamic array) to store all the zombies currently on the field and all the bullets in flight. This makes it easy to use a loop ("for" or "while") to update their positions every frame.

4.3. Collision detection

Because Plants vs. Zombies operate on separate horizontal rows, the collision logic is very simple. Therefore, you only need to consider objects in the same row. If the bullet's X-coordinate is greater than or equal to the zombie's X-coordinate, it counts as a collision and health is deducted.

4.4. Spawner algorithm

The zombie spawn system randomly selects one of five rows for zombies to appear in, based on a timer. This doesn't mean only a single zombie will appear each wave; there could be more than one zombie in each row or in the same row.

5. CONCLUSION

5.1. Achieved

Plants vs. Zombies successfully captivated players with its simple yet engaging gameplay that challenged their abilities. From calculating resource needs to predicting and strategically arranging plants to fight off zombie hordes, the game helped players develop logical thinking, improve their reaction time, and hone their organizational skills. Ultimately, after successfully defending their home from the zombie horde, players felt a sense of satisfaction and a desire to challenge themselves with more difficult levels - a key factor in the success and popularity of this tower defense game.

5.2. Challenges and solutions

Balancing the game is a difficult challenge. To prevent it from becoming too easy or too difficult, the amount of resources and the cost of different plant species must be proportionate. This also depends partly on which plants the player chooses to use in a given level; therefore, the plants need to complement each other (for example, Wall-nuts can tank damage for Peashooters). Furthermore, optimizing memory and RAM usage is crucial to prevent game crashes if there are too many objects on the battlefield. One of the solutions our team came up with is using ArrayList and safely removing elements via standard loop indexing (e.g., iterating backwards or decrementing indices) to optimize game performance and avoid ConcurrentModificationException across all configurations.